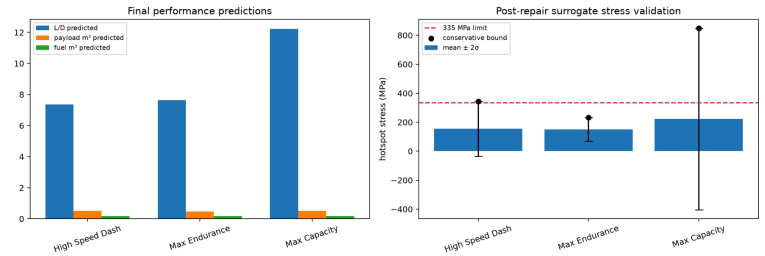


Technical Summary: Inverse Design Pipeline

This technical summary describes the inverse design optimization workflow. The objective here was to minimize a loss function while ensuring that the maximum stress at a hotspot did not exceed 335 MPa. Before discussing the current optimization model and why I made those choices, I'll start with my previous designs, working from the ground up.

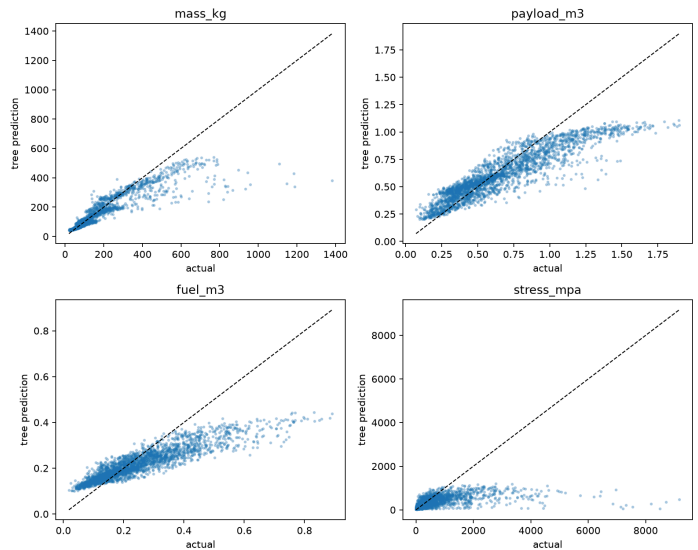
Dataset Row Bayesian Optimization Baseline

This was the simplest optimizer version, which searched for designs present in the dataset. I estimated loss through the 21 design variables with a Gaussian Process model. The optimization pipeline started with 0% exploitation and 100% exploration tolerance, but eventually (after ~100 trials) shifted to 50% exploitation and 50% exploration. During the optimization, L/D was recalculated using the supplied surrogate, while maxx, volume, and stress came from the selected row. So this version couldn't generate novel geometries. Because of this limitation, this version of the optimization pipeline was susceptible to confidence-interval issues, reaching as high as 800 MPa for some problems.

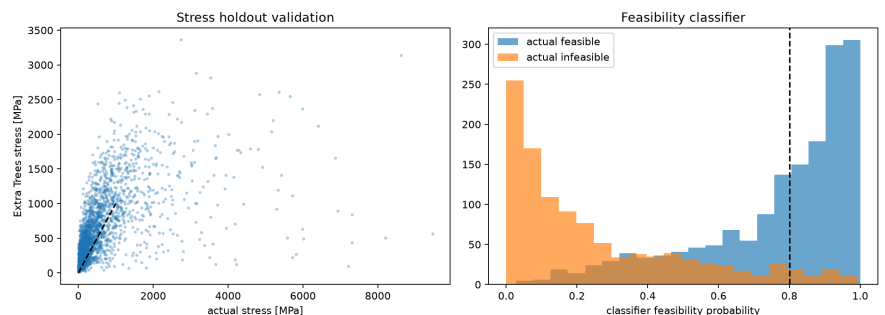


Ensemble Surrogate with Constrained Bayesian Optimization

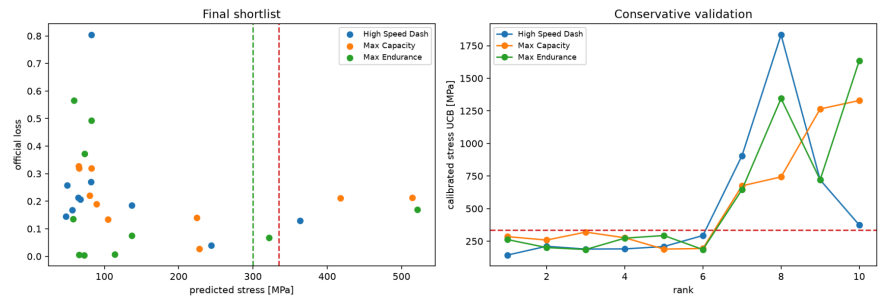
My next version replaced the row lookup with an eight-member bootstrap ensemble of random Fourier feature ridge models. These models predict the four structural outputs: empty mass, payload volume, fuel volume, and hotspot stress from design variables and the flight conditions. Training uses an 80/20 stress-stratified split, and the candidate search uses differentiable evolution for better-constrained expected improvement. Designs are clipped to bounds and repaired to obey discrete rib and spar rules, and are then checked across all ensemble members a final time. While this model was good for a quick but decent search, a model validation check showed that this ensemble surrogate missed some key outputs when compared to the dataset.



Feasibility First Hybrid Optimizer

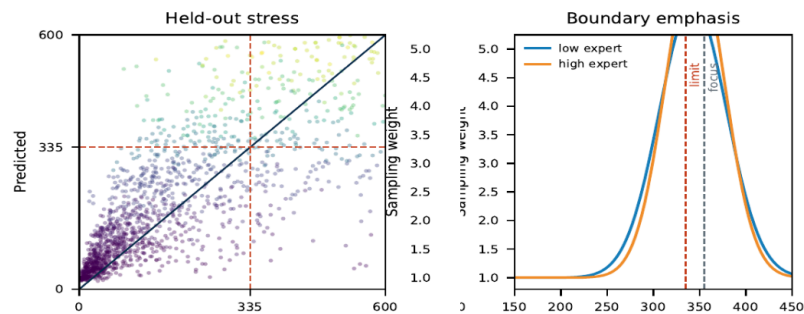


This first iteration used an external library (scikit-learn) to change the surrogate models and the search strategy. Four Extra Trees regressors predict the structural targets so that high-magnitude stress does not dominate the other outputs, and a separate Extra Trees classifier predicts whether a design is stress-feasible. We then use a holdout-calibrated stress upper confidence bound, along with a minimum 80% feasibility, to ensure the design does not violate the stress constraint. The search starts by selecting dataset rows below 300 MPa, then uses differential evolution to explore promising basins; it applies Gaussian Process Bayesian refinement.

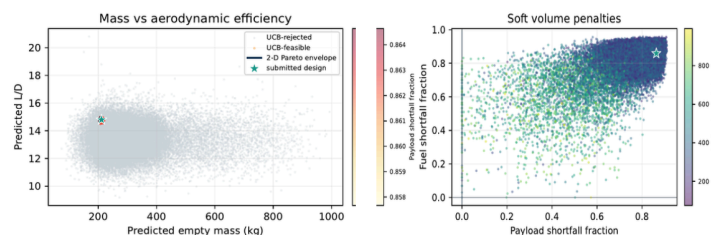


Coupled Stress Expert Optimizer.

This is my current version, so I can spend some more time on this optimization version. The goal was to avoid the stress limitations of the previous models by adding a specific stress surrogate to the ensemble, made up of a low-stress and high-stress surrogate. These stress surrogates are technically coupled because they overlap between 335 and 355 MPa, the region in the previous model with some of the more challenging optimizations. As in previous models, volume targets are soft constraints, so the optimizer may retain a lighter stress-safe design even when it misses a volume target. Stress is the hard constraint: a candidate is accepted only when its calibrated one-sided upper confidence bound (UCB) is at or below 335 MPa. Mass and volumes use bootstrap ensembles.



One trade-off I had to make in this final model was optimizing the max-capacity mission. The candidate cloud below is scored by the conservative UCB rule; points are surrogate predictions, not FE re-evaluations. The Pareto envelope is computed only among UCB-feasible candidates. This separates the mass/L/D comparison from the volume shortfalls, which remain part of the loss rather than pass/fail gates.



The coupled stress optimizer uses a similar structure to previous models with a differentiable evolution pass, but it then optimizes around the space using the loss function, checking for feasibility (max stress UCB) at each iteration. I had the idea of using the loss function when developing this optimization pipeline, so I could only compare it to past walks through the design space; however, in my experience, using the loss function has let me optimize a bit faster and get closer to a minimum.

